

# SNOMED CT 기반 Graph RAG 의료 질의응답 시스템 개발

2026 전기 졸업과제 중간보고서



팀 치즈브라우니

202255536 김채은 [chloe107@pusan.ac.kr](mailto:chloe107@pusan.ac.kr)

202255560 신우숨 [gee0209@pusan.ac.kr](mailto:gee0209@pusan.ac.kr)

202355647 이정현 [junghyeon4795@gmail.com](mailto:junghyeon4795@gmail.com)

지도교수: 송길태

분과명: 소프트웨어/인공지능

# 목차

## 1. 요구조건 및 제약 사항 분석에 대한 수정사항

- 1-1 요구조건 수정사항
- 1-2 제약 사항 분석 수정사항

## 2. 설계 상세화

### 2-1 시스템 전체 구조

- 2-1-1 시스템 구조도
- 2-1-2 전체 처리 흐름
- 2-1-3 개발 도구 및 기술

### 2-2 백엔드 설계

- 2-2-1 Neo4j Schema
- 2-2-2 Description Search 및 Entity Linking
- 2-2-3 Neo4j Query 및 Graph Context 생성
- 2-2-4 Guideline 구축
- 2-2-5 FastAPI 구조

### 2-3 프론트엔드 설계

- 2-3-1 설계 목표
- 2-3-2 화면 구성
- 2-3-3 컴포넌트 구조

## 3. 갱신된 과제 추진 계획

## 4. 구성원별 진척도

## 5. 과제 수행 내용 및 중간 결과

# 1. 요구조건 및 제약 사항 분석에 대한 수정사항

## 1-1. 요구조건 수정사항

### 1-1-1. 대상 질환 범위 변경

기존에는 만성질환(3대 성인병)과 중증질환(암)을 대상으로 시스템을 개발할 계획이었다.

그러나 의료 지식 그래프 구축과 임상 진료지침 확보의 효율성을 고려하여 다음의 5개 대표 만성질환으로 프로젝트 범위를 조정하였다.

- Type 2 Diabetes Mellitus
- Hypertension
- Chronic Kidney Disease
- Chronic Heart Failure
- Chronic Obstructive Pulmonary Disease(COPD)

### 1-1-2. 의료 온톨로지 구축 범위 확대

기존에는 SNOMED CT를 중심으로 의료 지식 그래프를 구축할 계획이었다.

그러나 보다 다양한 의료 정보를 활용하기 위해 SNOMED CT뿐만 아니라 RxNorm과 LOINC까지 함께 구축하도록 요구사항을 수정하였다.

### 1-1-3. Description Search 및 Entity Linking 요구사항 구체화

기존에는 키워드 기반 또는 임베딩 기반 분석만 정의하였다.

이를 BM25 기반 Keyword Search와 Embedding Search를 결합한 Hybrid Search 방식으로 구체화하였으며, Hybrid Score와 Mention Detection의 threshold를 적용하여 Entity Linking의 정확도를 향상시키도록 요구사항을 보완하였다.

### 1-1-4. Graph 탐색 기능 구체화

기존에는 Multi-hop Graph 탐색만 정의하였다.

이를 Concept Set을 시작점으로 최대 3-Hop까지 탐색하도록 구체화하였으며, 허용 Relationship 제한과 질문 유형에 따른 Dynamic Query 기능을 추가하였다.

### 1-1-5. Guideline Retrieval 기능 추가

기존에는 Graph Context 생성 기능만 정의하였다.

이를 WHO와 NICE의 Clinical Guideline을 Qdrant에 구축하고, Graph Context와 함

게 검색하여 LLM 입력에 활용하는 구조로 확장하였다. 이를 통해 의료 지식 그래프와 임상 진료지침을 함께 활용하는 근거 기반 질의응답이 가능하도록 요구사항을 보완하였다.

#### **1-1-6. 시스템 구조 모듈화**

유지보수성과 확장성을 높이기 위해 FastAPI 기반의 모듈형 구조를 적용하였다. EntityLinkingService, GraphService, GuidelineService, LLMService를 독립적으로 구성하고, ChatService가 전체 처리 과정을 관리하도록 요구사항을 구체화하였다.

### **1-2. 제약 사항 분석 수정사항**

#### **1-2-1. 의료 온톨로지 데이터 규모**

SNOMED CT는 수십만 개 이상의 Concept와 복잡한 Relationship를 포함하고 있어 전체 데이터를 그대로 활용할 경우 그래프 탐색 성능이 저하될 수 있다.

이를 해결하기 위해 프로젝트 대상 질환과 직접 관련된 Concept를 중심으로 의료 지식 그래프를 구축하고, Graph 탐색 시 허용된 Relationship와 최대 3-Hop 제한을 적용하여 탐색 범위를 최적화하였다.

#### **1-2-2. 자연어와 의료 Concept 간의 매핑 정확도**

사용자의 질문에는 약어, 동의어, 비표준 표현 등이 포함될 수 있으므로 단순 Keyword Search만으로는 정확한 Concept를 찾기 어렵다.

이를 해결하기 위해 BM25 기반 Keyword Search와 Embedding Search를 결합한 Hybrid Entity Linking을 적용하고, Hybrid Score와 Mention Detection의 threshold를 함께 사용하여 Concept 매핑의 정확도를 향상시키도록 설계하였다.

#### **1-2-3. Guideline 검색 정확도**

Clinical Guideline은 문서의 길이가 길고 다양한 질환 정보를 포함하고 있어 단순 Embedding Search만 수행할 경우 관련성이 낮은 문서를 검색할 가능성이 있다.

이를 해결하기 위해 Guideline을 Chunk 단위로 분할하고 Metadata를 함께 저장하여 Metadata Filter와 Embedding Search를 순차적으로 수행하도록 설계하였다. 이를 통해 검색 범위를 줄이고 검색 정확도를 향상시킬 수 있도록 하였다.

#### **1-2-4. LLM의 Hallucination 문제**

LLM은 의료 분야에서 실제 근거와 다른 정보를 생성할 가능성이 존재한다.

이를 최소화하기 위해 Neo4j에서 생성한 Graph Context와 Clinical Guideline 검색 결과를 함께 Prompt에 제공하여 LLM이 근거 기반 정보를 활용하도록 설계하였다.

### 1-2-5. 시스템 확장성

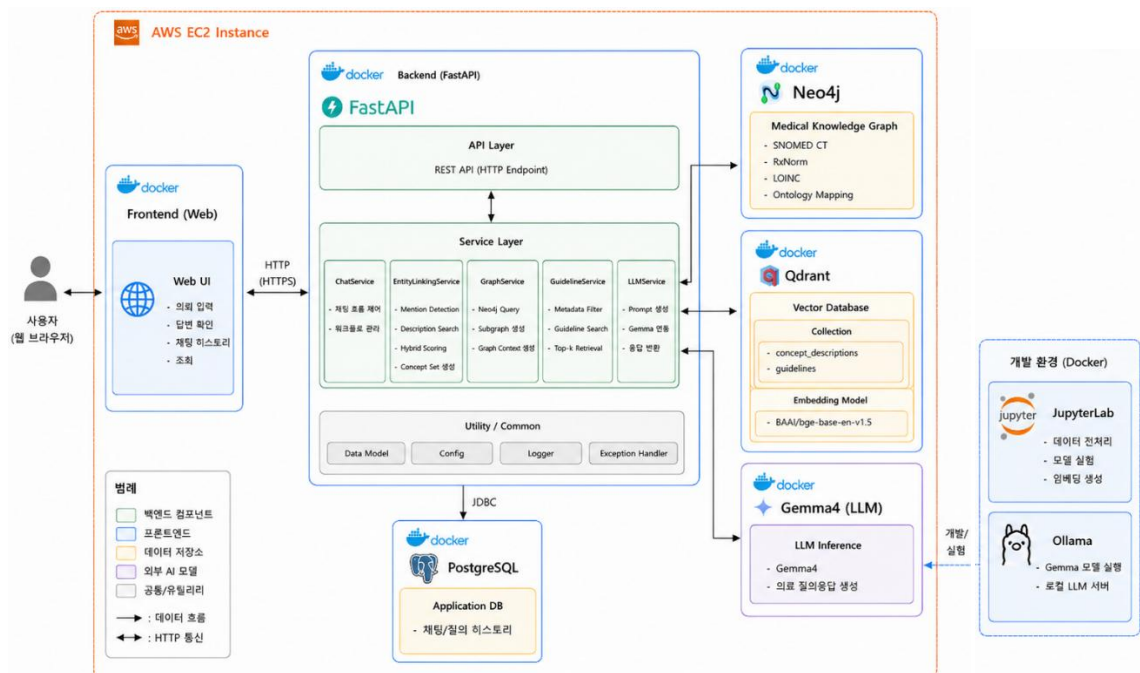
향후 새로운 질환이나 의료 온톨로지를 추가할 경우 기존 시스템을 크게 수정하지 않고 확장할 수 있어야 한다.

이를 위해 Neo4j와 Qdrant를 독립적으로 구성하고, FastAPI 기반의 모듈형 구조를 적용하여 새로운 질환, Guideline, Ontology를 손쉽게 추가할 수 있도록 설계하였다.

## 2. 설계 상세화

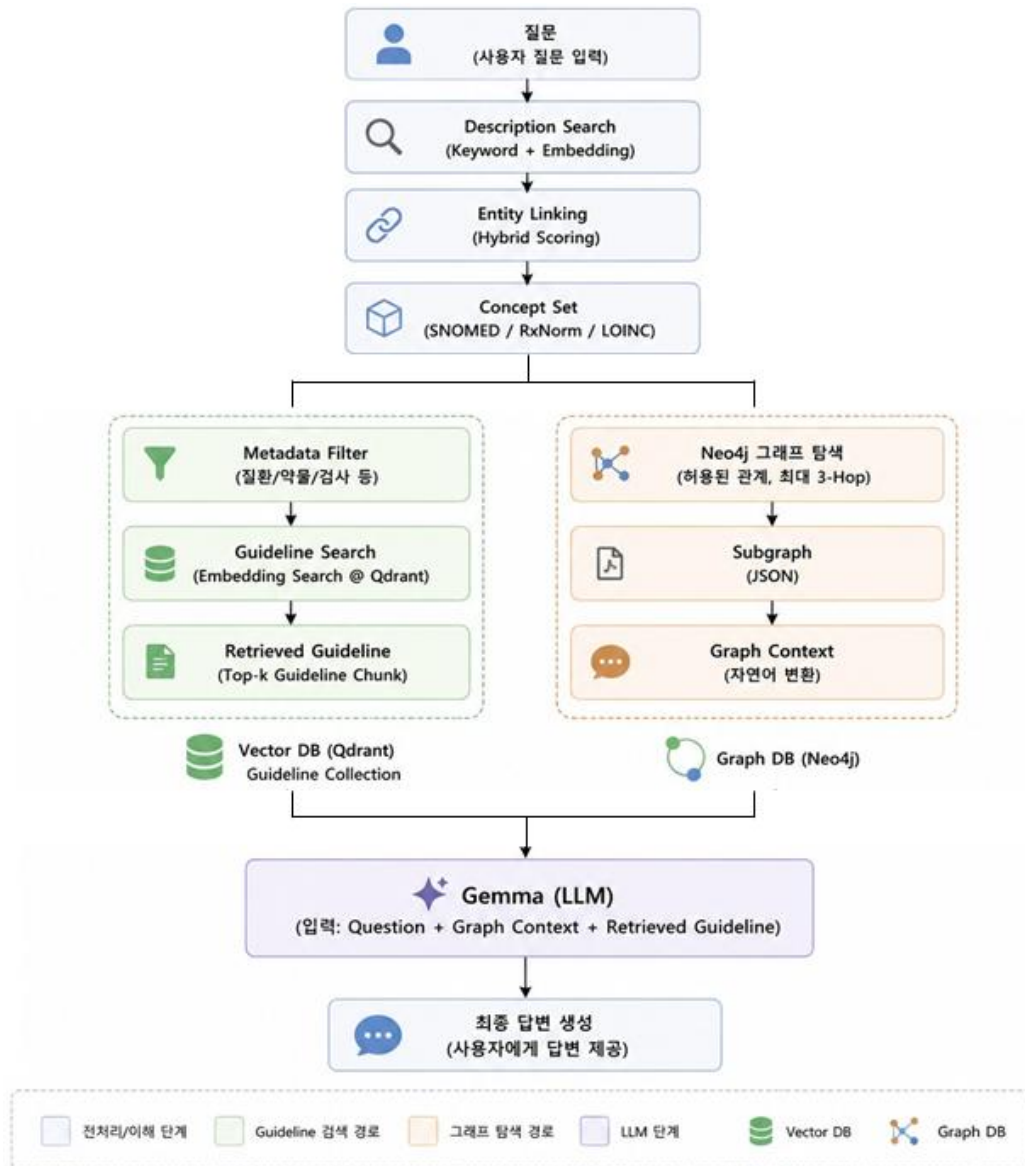
### 2-1. 시스템 전체 구조

#### 2-1-1. 시스템 구조도



사용자가 웹 인터페이스를 통해 질문을 입력하면 FastAPI가 요청을 받아 Entity Linking, Graph Query, Guideline Search, LLM 추론 등의 서비스를 순차적으로 수행한다. 의료 지식 그래프는 Neo4j에, 의료 개념 및 임상 진료지침 임베딩은 Qdrant에 저장되며, 최종적으로 Graph Context와 Guideline 정보를 함께 활용하여 Gemma4 모델이 근거 기반 의료 답변을 생성한다. 모든 구성 요소는 Docker 컨테이너 환경에서 독립적으로 운영되도록 설계하였다.

## 2-1-2. 전체 처리 흐름



사용자의 질문은 Description Search와 Entity Linking을 통해 의료 Concept Set으로 변환되며, 이를 기반으로 Neo4j에서는 의료 지식 그래프를 탐색하여 Graph Context를 생성하고, Qdrant에서는 관련 Clinical Guideline을 검색한다. 이후 사용자 질문 (Question), Graph Context, Retrieved Guideline을 함께 Gemma 모델의 입력으로 활용하여 근거 기반의 최종 답변을 생성하고 사용자에게 제공한다.

이를 통해 단순 LLM 기반 질의응답보다 근거 기반의 의료 정보를 제공하도록 설계하였다.

2-1-3. 개발 도구 및 기술

| 구성 요소                | 사용 기술                          |
|----------------------|--------------------------------|
| Front-end            | React (Vite)                   |
| Back-end             | FastAPI                        |
| Programming Language | Python                         |
| Relational Database  | PostgreSQL                     |
| LLM                  | Gemma4                         |
| Graph Database       | Neo4j                          |
| Vector Database      | Qdrant                         |
| Embedding Model      | BAAI/bge-base-en-v1.5          |
| Keyword Search       | BM25 (rank_bm25)               |
| Mention Detection    | SciSpacy (en_core_sci_scibert) |
| Container            | Docker Compose                 |

본 시스템은 FastAPI를 중심으로 백엔드 서비스를 구성하고, React(Vite)를 이용하여 사용자 인터페이스를 구현하였다. 의료 지식 그래프는 Neo4j에 저장하고, 임베딩 데이터 및 Clinical Guideline은 Qdrant에 구축하였다. Gemma4는 Ollama 환경에서 구동하며, Docker Compose를 이용하여 주요 서비스를 컨테이너 환경으로 관리하도록 구성하였다.

2-2. 백엔드 설계

2-2-1. Neo4j Schema

의료 지식 그래프는 SNOMED CT, RxNorm, LOINC의 세 가지 온톨로지를 하나의 그래프에서 통합 관리할 수 있도록 Neo4j 스키마를 설계하였다.

모든 노드는 concept\_id, name, ontology, semantic\_type의 공통 속성을 포함하도록 설계하였다. 또한 각 온톨로지의 특성을 반영하기 위해 RxNorm에는 TTY(Term Type), LOINC에는 NAME\_SOURCE를 추가하여 저장하도록 설계하였다.

※ 아래 표에는 모든 노드가 공통적으로 가지는 속성만 나타내었다.

| 속성            | 설명                         |
|---------------|----------------------------|
| concept_id    | 각 온톨로지의 고유 식별자             |
| name          | 의료 개체의 명칭                  |
| ontology      | SNOMED CT / RxNorm / LOINC |
| semantic_type | 질병, 약물, 검사 등 개체 유형         |

공통 속성을 기반으로 서로 다른 온톨로지를 동일한 방식으로 조회할 수 있으며, 온톨로지별 속성을 활용하여 보다 세부적인 검색과 기능 확장이 가능하도록 설계하였다.

## 2-2-2. Description Search 및 Entity Linking

Description Search는 입력된 질문과 가장 유사한 의료 Concept를 검색하여 Entity Linking을 수행하는 단계로, 이후 Neo4j 그래프 탐색의 시작점이 되는 Concept Set을 생성한다.

Description Search는 Keyword Search와 Embedding Search를 함께 사용하는 Hybrid Search 방식을 적용하였다. Keyword Search는 용어의 정확한 일치 여부를 반영하는 반면, Embedding Search는 의미적으로 유사한 표현까지 검색할 수 있어 두 방법의 장점을 동시에 활용할 수 있다.

### (1) Mention Detection

사용자 질문으로부터 질병, 약물, 검사명 등 의료 개체를 추출하는 과정이다. SciSpacy의 en\_core\_sci\_scibert 모델을 사용하여 의료 개체를 추출하며, 동일한 Mention은 중복 제거 후 Keyword Search와 Embedding Search의 입력으로 사용한다.

### (2) Keyword Search

BM25 알고리즘을 사용하여 검색을 통해 질문과 키워드가 일치하는 Description을 검색하고 상위 후보 Concept를 추출한다.

### (3) Embedding Search

질문과 Description을 동일한 벡터 공간으로 변환한 후 의미적 유사도를 계산하여 후보 Concept를 검색한다.

Embedding 모델은 BAAI/bge-base-en-v1.5를 사용하도록 설계하였다.

Embedding된 Description은 Qdrant의 concept\_descriptions Collection에 저장하며, 각 데이터는 conceptId, term, descriptionType, ontology, score를 Payload로 저장한다.

#### (4) Hybrid Search

Keyword Search와 Embedding Search의 결과를 결합하기 위해 Hybrid Score를 계산한다. 우선 BM25 검색 결과 상위 10개와 Embedding 검색 결과 상위 10개를 각각 추출한 후 이를 통합하여 최종 점수를 계산한다.

Hybrid Score는 다음과 같이 계산하도록 설계하였다.

➔  $\text{Final Score} = 0.2 \times \text{BM25 Score} + 0.8 \times \text{Embedding Score}$

Embedding Search가 의미적 유사성을 더 잘 반영하므로 높은 가중치를 부여하였으며, 최종 가중치는 실험을 통해 가장 높은 성능을 보이는 값으로 조정할 예정이다.

#### (5) Entity Linking

Mention Detection을 통해 추출된 각 Mention에 대해 Hybrid Search를 수행하여 Top-3 Concept를 검색하고, 이를 Concept Set으로 구성하여 Neo4j 그래프 탐색의 입력으로 사용한다.

### 2-2-3. Neo4j Query 및 Graph Context 생성

Description Search를 통해 생성된 Concept Set은 Neo4j 그래프 탐색의 시작점으로 사용된다. 본 시스템에서는 질문에서 추출된 모든 Concept를 시작점으로 하여 각각 최대 3-Hop까지 그래프를 탐색하도록 설계하였다. 이를 통해 질병, 약물, 검사 등 의료 개체 간의 관계를 종합적으로 탐색하고, 이후 Graph Context 생성에 필요한 Subgraph를 구축한다.

#### (1) Graph 탐색 전략

Graph 탐색 시에는 모든 Relationship를 사용하는 것이 아니라, 의료 질의응답에 필요한 주요 Relationship와 프로젝트에서 정의한 Ontology Mapping 관계만을 선택적으로 활용하도록 설계하였다. 이를 통해 불필요한 탐색을 줄이고 의료적으로 의미 있는 정보만 추출할 수 있도록 하였다.

| Relationship          | 활용 목적             |
|-----------------------|-------------------|
| IS_A                  | 질병 및 개념의 계층 구조 탐색 |
| FINDING_SITE          | 질병 발생 부위          |
| DUE_TO                | 원인 질환             |
| INTERPRETS            | 검사 결과의 해석 대상      |
| CAUSATIVE_AGENT       | 원인 병원체            |
| ASSOCIATED_MORPHOLOGY | 관련 형태학적 변화        |
| OCCURRENCE            | 발생 시기             |
| HAS_DRUG              | 질병과 치료 약물 연결      |
| HAS_TEST              | 약물과 관련 검사 관계      |

현재 활용하는 Relationship는 대표 질환과의 연관성이 높은 관계를 중심으로 선정한 초기 설계안이며, Graph Query 성능 평가 결과에 따라 지속적으로 보완할 예정이다.

## (2) Dynamic Query 설계

사용자의 질문 의도에 따라 탐색해야 하는 Relationship는 달라질 수 있으므로, 본 시스템에서는 Dynamic Query 방식을 적용하였다.

예시는 다음과 같다.

| 질문 유형 | 우선 탐색 관계        |
|-------|-----------------|
| 약물 추천 | HAS_DRUG        |
| 검사 추천 | HAS_TEST        |
| 합병증   | ASSOCIATED_WITH |

## (3) Graph Context 생성

Neo4j에서 검색된 결과는 Subgraph(JSON) 형태로 반환되지만, JSON 데이터는 LLM이 직접 이해하기 어렵다. 따라서 본 시스템에서는 Subgraph를 자연어 형태의 Graph Context로 변환하는 과정을 함께 수행한다.

Graph Context Generator는 규칙 기반(Template-based) 방식으로 설계하였다. 각 Relationship마다 문장 템플릿을 정의하고, 조회된 관계에 따라 자동으로 자연어

문장을 생성한다.

대표적인 템플릿 예시는 다음과 같다.

| Relationship          | 생성 문장 예시                                            |
|-----------------------|-----------------------------------------------------|
| IS_A                  | {A} is a type of {B}                                |
| FINDING_SITE          | {A} occurs at {B}                                   |
| DUE_TO                | {A} may be caused by {B}                            |
| INTERPRETS            | {A} is assessed by evaluating {B}                   |
| ASSOCIATED_MORPHOLOGY | {A} is associated with the morphological change {B} |
| CAUSATIVE_AGENT       | {A} is caused by the agent {B}                      |
| OCCURRENCE            | {A} occurs during {B}                               |
| HAS_DRUG              | {A} may be treated with {B}                         |
| HAS_TEST              | {A} should be monitored using {B}                   |

이처럼 생성된 Graph Context는 질문과 함께 Guideline 검색 결과와 결합되어 Gemma4의 입력으로 사용된다. JSON 형태의 그래프를 그대로 전달하는 것보다 자연어 문장으로 변환하여 제공하면 LLM이 의료 개체 간의 관계를 보다 정확하게 이해할 수 있으며, 최종 답변의 정확성과 추론 능력을 향상시킬 수 있다.

#### 2-2-4. Guideline 구축

본 시스템은 의료 지식 그래프뿐만 아니라 임상 진료지침(Clinical Guideline)을 함께 활용하는 GraphRAG 구조를 적용하였다. Neo4j는 질병, 약물, 검사 간의 관계를 제공하는 반면, 실제 치료 권고사항과 임상적 근거는 WHO, NICE의 Clinical Guideline에 포함되어 있다. 따라서 정확하고 근거 기반의 의료 질의응답을 위해 Guideline을 Vector Database(Qdrant)에 구축한다.

##### (1) Guideline 전처리

대표 질환에 대한 Clinical Guideline(PDF)에서 텍스트를 추출한 후 검색에 적합한 형태로 전처리를 수행한다. PDF의 불필요한 서식이나 페이지 번호 등을 제거하고 순수한 본문만 추출하여 이후 Chunk 생성 과정에 사용한다.

## (2) Chunk 생성

추출한 Guideline은 그대로 저장하지 않고 일정한 단위의 Chunk로 분할하여 저장한다. Recommendation이나 Section과 같이 문서 자체에 의미 있는 구분이 존재하는 경우에는 해당 단위를 하나의 Chunk로 사용한다.

반면 Recommendation이 지나치게 길거나 문서에 별도의 구분이 없는 경우에는 약 300~500 Tokens 단위로 분할하며, 평균 400 Tokens를 기준으로 Chunk를 생성한다. 또한 앞뒤 문맥이 자연스럽게 이어질 수 있도록 80 Tokens 정도의 Overlap을 적용하여 인접한 Chunk 간 일부 내용을 공유하도록 설계하였다.

이와 같은 Chunking 방식은 문맥 손실을 최소화하면서도 의미 단위가 유지되도록 하기 위한 것이다.

## (3) Metadata 구성

각 Guideline Chunk에는 검색 정확도를 높이기 위한 Metadata를 함께 저장하여 향후 Vector Search 이전에 Metadata Filter를 수행할 수 있도록 설계하였다.

## (4) Guideline Retrieval

전처리가 완료된 Guideline Chunk는 BAAI/bge-base-en-v1.5 모델을 이용하여 Embedding을 생성한 후 Qdrant의 guidelines Collection에 저장한다.

질문이 입력되면 Description Search와 Entity Linking을 통해 생성된 Concept Set을 이용하여 먼저 Metadata Filter를 수행한다. 이후 Filter를 통과한 Guideline에 대해 Embedding Search를 수행하여 의미적으로 가장 유사한 Top-k Chunk를 검색한다.

검색 과정은 다음과 같다.

➔ Question → Concept Set → Metadata Filter → Embedding Search → Top-k Guideline Chunk

Graph Context에서 추출된 질병, 약물, 검사 정보 역시 Guideline 검색에 함께 활용한다. 이를 통해 단순한 의미 검색보다 검색 성능을 향상시킬 수 있다.

최종적으로 검색된 Top-k Guideline은 Graph Context와 함께 Gemma4의 입력으로 사용된다.

### 2-2-5. FastAPI 구조

전체 시스템은 FastAPI를 중심으로 모듈화하여 설계하였다. 각 기능은 독립적인 Service 형태로 구현하며 ChatService가 전체 흐름을 관리한다.

전체 처리 과정은 다음과 같다.

➔ 질문 수신 → ChatService → EntityLinkingService → GraphService → GuidelineService → LLMService → Gemma4 → 응답 반환

이와 같은 구조는 기능별 역할을 명확하게 분리하여 유지보수와 기능 확장이 용이하도록 설계하였다.

### **(1) ChatService**

ChatService는 전체 질의응답 과정을 관리하는 Controller 역할을 수행한다.

사용자로부터 질문을 입력받으면 Entity Linking, Graph 검색, Guideline 검색, LLM 호출을 순차적으로 실행하고, 생성된 최종 답변을 사용자에게 반환한다. ChatService는 각 Service를 호출하여 처리 순서를 관리하는 역할만 수행하며, 실제 데이터 처리 로직은 포함하지 않는다.

### **(2) EntityLinkingService**

EntityLinkingService는 Description Search를 수행하여 질문에서 의료 Concept를 추출한다.

BM25 기반 Keyword Search와 Embedding Search 결과를 Hybrid Score로 결합하여 최종 Concept Set을 생성하며, 이후 GraphService와 GuidelineService에서 사용할 의료 개체 정보를 제공한다.

### **(3) GraphService**

GraphService는 Concept Set을 입력받아 Neo4j에서 Subgraph를 검색하고, Graph Context Generator를 이용하여 자연어 형태의 Graph Context를 생성한다.

Neo4j에서는 단순히 그래프를 조회하는 기능만 수행하며, Graph Context 생성은 GraphService 내부에서 수행하도록 설계하였다. 따라서 Graph Context의 문장 템플릿을 수정하는 경우에도 Neo4j 검색 로직은 변경하지 않아도 되므로 유지보수성이 향상된다.

GraphService의 내부 처리 과정은 다음과 같다.

➔ Concept Set → Neo4j Search → Subgraph 생성 → Template 적용 → Graph Context 생성

#### **(4) GuidelineService**

GuidelineService는 Entity Linking에서 생성된 Concept Set과 Graph Context를 활용하여 Guideline을 검색한다.

먼저 Metadata Filter를 수행한 후 Embedding Search를 실행하여 의미적으로 가장 유사한 Top-k Guideline Chunk를 검색하며, 검색 결과를 LLMService에 전달한다.

#### **(5) LLMService**

LLMService는 Question, Graph Context, Retrieved Guideline을 하나의 Prompt로 구성하여 Gemma4에 전달한다.

최종 Prompt는 다음과 같은 형태로 구성된다.

- 사용자 질문(Question)
- Graph Context
- Retrieved Guideline(Top-k)

Gemma4는 입력된 Prompt를 바탕으로 의료 지식 그래프와 임상 진료지침을 모두 반영한 최종 답변을 생성하며, 생성된 답변은 FastAPI를 통해 사용자에게 반환된다.

### **2-3. 프론트엔드 설계**

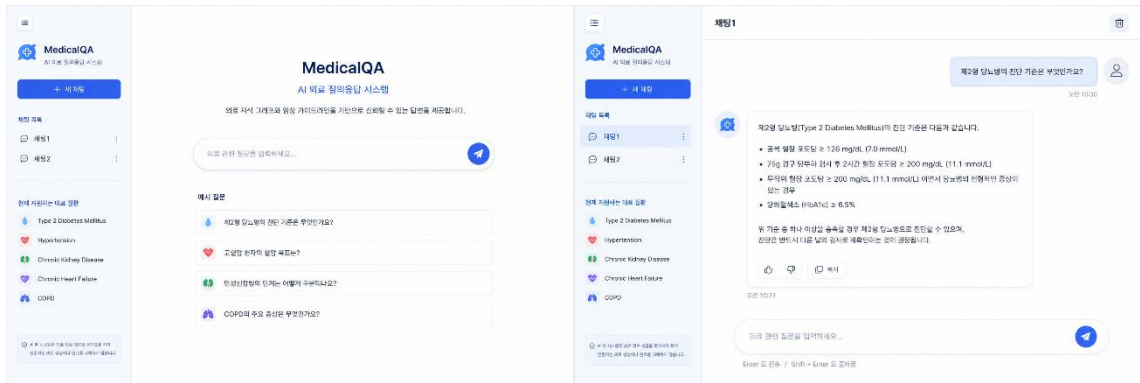
#### **2-3-1. 설계 목표**

사용자가 의료 질의응답 서비스를 직관적으로 이용할 수 있도록 채팅 기반 인터페이스로 설계하였다. 질문 입력, 답변 확인, 채팅 히스토리 조회 기능을 하나의 화면에서 제공하며, FastAPI 백엔드와 연동하여 질의응답을 수행한다. 또한 컴포넌트 기반 구조를 적용하여 유지보수성과 기능 확장성을 고려하였다.

#### **2-3-2. 화면 구성**

Sidebar 영역과 Chat 영역으로 구성된 단일 페이지 구조로 설계하였다. Sidebar에서는 새로운 대화를 생성하고 이전 대화 기록을 조회할 수 있으며, Chat 영역에서는 질문 입력과 답변 확인이 이루어진다. 초기 화면에서는 시스템 소개와 예시 질문을 제공하여 사용자가 서비스의 기능을 쉽게 이해하고 바로 질의를 시작할 수 있도록 구성하였다.

화면 구성 예시는 다음과 같다.



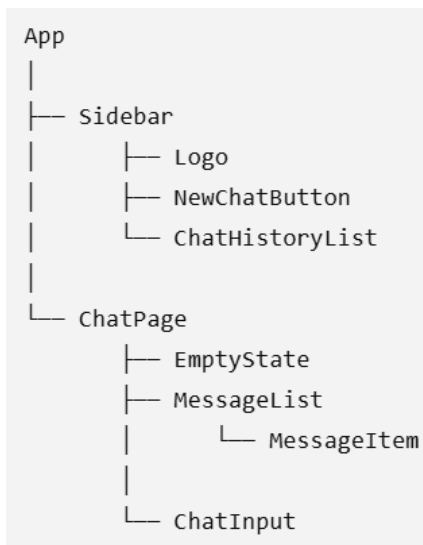
화면의 주요 구성 요소는 다음과 같다.

| 구성 요소        | 기능                     |
|--------------|------------------------|
| Sidebar      | 새로운 대화 생성 및 채팅 히스토리 조회 |
| Chat Area    | 질문 및 답변 표시             |
| Input Box    | 자연어 질문 입력              |
| Send Button  | 질문 전송                  |
| Message View | 생성된 답변 출력              |

### 2-3-3. 컴포넌트 구조

화면의 유지보수성과 재사용성을 고려하여 컴포넌트 기반 구조로 설계하였다. 각 화면 요소를 독립적인 컴포넌트로 분리함으로써 향후 기능 추가 및 UI 변경 시 특정 컴포넌트만 수정할 수 있도록 하였다.

전체 컴포넌트 구조는 다음과 같다.



각 컴포넌트의 역할은 다음과 같다.

| 컴포넌트            | 역할                          |
|-----------------|-----------------------------|
| App             | 전체 페이지 레이아웃 관리              |
| Sidebar         | 새로운 대화 생성 및 채팅 히스토리 관리      |
| Logo            | 시스템 이름(Medical QA) 표시       |
| NewChatButton   | 새로운 채팅 생성                   |
| ChatHistoryList | 이전 대화 목록 표시                 |
| ChatPage        | 채팅 화면 전체 구성 및 상태 관리         |
| EmptyState      | 대화가 없는 초기 화면 표시 및 안내 메시지 출력 |
| MessageList     | 질문과 답변 목록 출력                |
| MessageItem     | 개별 질문 및 답변 표시               |
| ChatInput       | 질문 입력 및 서버 전송               |

초기 화면에서는 EmptyState 컴포넌트를 통해 시스템 안내 문구와 질문 입력창을 화면 중앙에 배치하여 사용자가 바로 질문을 입력할 수 있도록 설계하였다. 사용자가 ChatInput에서 질문을 입력하면 FastAPI를 호출하여 답변을 요청하고, 생성된 답변은 MessageList에 추가되어 대화 형식으로 출력된다. 이후 초기 화면은 대화 화면으로 전환되며, 질문과 답변은 데이터베이스에 저장된다. 저장된 대화는 Sidebar의 ChatHistoryList를 통해 조회할 수 있으며, NewChatButton을 이용하여 새로운 대화를 시작할 수 있도록 구성하였다.

### 3. 갱신된 과제 추진 계획

| 구분                        | 세부 항목                        | 5월 | 6월 | 7월 | 8월 | 9월 |
|---------------------------|------------------------------|----|----|----|----|----|
| 기획                        | 기능 구체화                       |    |    |    |    |    |
|                           | 대상 질환 선정                     |    |    |    |    |    |
| 개발환경 구축                   | Docker 기반 개발환경 구축            |    |    |    |    |    |
|                           | Neo4j 및 Qdrant 구축            |    |    |    |    |    |
|                           | React(Vite) 개발환경 구축          |    |    |    |    |    |
| 의료 지식<br>그래프구축            | 의료 온톨로지 데이터 수집               |    |    |    |    |    |
|                           | 온톨로지 데이터 전처리                 |    |    |    |    |    |
|                           | 온톨로지 간 매핑(UMLS)              |    |    |    |    |    |
|                           | Neo4j 스키마 설계                 |    |    |    |    |    |
|                           | Neo4j Node 및 Relationship 구축 |    |    |    |    |    |
|                           | HAS_DRUG/HAS_TEST 관계 구축      |    |    |    |    |    |
|                           | Graph Query 설계               |    |    |    |    |    |
|                           | Graph Context 생성             |    |    |    |    |    |
| Entity Linking<br>개발      | Mention Detection 구현         |    |    |    |    |    |
|                           | Embedding Search 구현          |    |    |    |    |    |
|                           | Keyword Search 구현            |    |    |    |    |    |
|                           | Hybrid Search 구현             |    |    |    |    |    |
|                           | Entity Linking 구현            |    |    |    |    |    |
| Guideline<br>Retrieval 개발 | Clinical Guideline 전처리       |    |    |    |    |    |
|                           | Guideline Embedding 구축       |    |    |    |    |    |
|                           | Guideline Retrieval 구현       |    |    |    |    |    |
| API 개발                    | FastAPI 서버 구축                |    |    |    |    |    |
|                           | Retrieval API 구현             |    |    |    |    |    |
|                           | LLM 응답 생성 API 구현             |    |    |    |    |    |
|                           | API 통합 및 테스트                 |    |    |    |    |    |
| 프론트엔드<br>개발               | 컴포넌트 구조 설계                   |    |    |    |    |    |
|                           | UI/UX 화면 구현                  |    |    |    |    |    |
|                           | API 연동                       |    |    |    |    |    |
|                           | PostgreSQL 기반 채팅 히스토리 관리     |    |    |    |    |    |
|                           | 통합 테스트 및 기능 검증               |    |    |    |    |    |
| 성능 평가 및<br>실험             | 질문 데이터셋 구축                   |    |    |    |    |    |
|                           | LLM vs RAG 비교 실험             |    |    |    |    |    |
|                           | 결과 분석 및 시각화                  |    |    |    |    |    |
| 최종 검증<br>및 보고서 작성         | 오류 수정 및 성능 개선                |    |    |    |    |    |
|                           | 보고서 작성                       |    |    |    |    |    |

## 4. 구성원별 진척도

| 이름  | 담당                                                                                                                                                                                                                                                                                                                                                                 |
|-----|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 김채은 | <ul style="list-style-type: none"><li>- Neo4j 및 Qdrant 환경 구축</li><li>- 의료 온톨로지(SNOMED CT, RxNorm, LOINC) 데이터 전처리</li><li>- SciSpacy 기반 Mention Detection 구현</li><li>- BAAI/bge-base-en-v1.5 기반 Embedding Search(Qdrant) 구현</li><li>- BM25 기반 Keyword Search 구현</li><li>- Hybrid Search 구현 및 검색 결과 통합</li><li>- Hybrid Search 기반 Entity Linking 구현 (진행 중)</li></ul> |
| 신우숨 | <ul style="list-style-type: none"><li>- React 개발 환경 구축</li><li>- 프론트엔드 컴포넌트 구조 설계</li><li>- 채팅 화면 레이아웃 및 UI 설계</li><li>- 채팅 페이지 구현 (예정)</li><li>- Sidebar 기능 구현 (예정)</li><li>- PostgreSQL 기반 채팅 히스토리 저장 및 조회 기능 구현 (예정)</li></ul>                                                                                                                                  |
| 이정현 | <ul style="list-style-type: none"><li>- SNOMED CT 노드 및 Relationship 적재</li><li>- LOINC 및 RxNorm 노드 적재</li><li>- HAS_DRUG, HAS_TEST Mapping 관계 적재</li><li>- Graph Query 설계 및 구현 (진행 중)</li><li>- Graph context 생성 (진행 중)</li><li>- RxNorm 및 LOINC 내부 Relationship 추가 (예정)</li></ul>                                                                                 |

## 5. 과제 수행 내용 및 중간 결과

### 5-1. 개발 환경 구축

의료 질의응답 시스템의 개발 환경 구축을 위해 Docker Compose를 활용하여 Neo4j Graph Database와 Qdrant Vector Database를 컨테이너 환경으로 구성하였다. 각 데이터베이스는 독립적인 컨테이너에서 실행되도록 구성하였으며, 데이터와 로그는 Volume을 이용해 영구적으로 저장되도록 설정하였다.

Neo4j는 의료 온톨로지 기반의 지식 그래프 저장 및 탐색을 담당하고, Qdrant는 임베딩된 의료 Concept를 저장하여 의미 기반 검색을 수행하도록 구축하였다. 또한 포트 매핑을 통해 외부 접근이 가능하도록 설정하여 이후 백엔드 및 검색 모듈과의 연동 환경을 마련하였다.

또한 React(Vite) 기반의 프론트엔드 개발 환경을 구축하고 프로젝트 초기 구조를 구성하였다. 컴포넌트 기반 디렉토리를 구성하여 화면 개발을 위한 기반을 마련하였으며, 이후 FastAPI와 연동할 수 있도록 프로젝트 환경을 구축하였다.

### 5-2. Entity Linking 모듈 구현

#### 5-2-1. 의료 온톨로지 데이터 전처리

Entity Linking에 활용하기 위해 SNOMED CT, RxNorm, LOINC 데이터를 전처리하였다. 각 온톨로지는 서로 다른 데이터 구조를 가지므로 검색 과정에서 동일하게 활용할 수 있도록 공통 데이터 형식으로 변환하였다.

SNOMED CT와 RxNorm은 대표 용어와 Synonym을 함께 활용하여 Description 데이터를 생성하였으며, LOINC는 Long Common Name, Short Name, Display Name, Consumer Name 등을 이용하여 다양한 Description을 추출하였다.

최종적으로 세 온톨로지의 데이터를 conceptId, term, descriptionType, ontology의 공통 컬럼 구조로 통합하였으며, 생성된 Description 데이터셋은 Embedding Search와 Keyword Search에서 공통으로 사용하도록 구성하였다.

#### 5-2-2. Mention Detection

사용자 질문에서 질병, 약물, 검사명과 같은 의료 개체를 추출하기 위해 Mention Detection 기능을 구현하였다.

모델 선정을 위해 SciSpacy(en\_core\_sci\_scibert)와 d4data/biomedical-ner-all 모델

을 동일한 질의에 적용하여 비교하였다.

Type 2 diabetes patients with hypertension should take metformin and monitor HbA1c regularly.

위와 같은 쿼리를 넣었을 때,

```
Predicted Entities
- type | History | score=0.5314
- 2 diabetes | Disease_disorder | score=0.9359
- hyper | History | score=0.8388
- ##tension | Disease_disorder | score=0.8731
- metformin | Medication | score=0.9994
- hba1c | Diagnostic_procedure | score=0.9646
```

➔ d4data/biomedical-ner-all모델 사용

```
Mentions: ['Type 2 diabetes', 'patients', 'hypertension', 'metformin', 'monitor', 'HbA1c']
```

➔ SciSpacy의 en\_core\_sci\_scibert 모델 사용

비교 결과 SciSpacy가 의료 개체를 보다 안정적으로 추출하였으며, 불필요한 Span 분할과 누락이 적은 것을 확인하였다. 이에 SciSpacy를 최종 Mention Detection 모델로 선정하였다.

또한 동일한 의료 개체가 중복 검출되는 경우에는 하나의 Mention만 유지하도록 후처리를 적용하였으며, 추출된 Mention은 이후 Entity Linking 과정의 입력으로 활용된다.

### 5-2-3. Embedding Search

전처리한 Description 데이터를 BAAI의 bge-base-en-v1.5 모델을 이용하여 임베딩한 후 Qdrant Vector Database에 저장하였다.

검색 시에는 사용자 질의를 동일한 방식으로 임베딩하여 Cosine Similarity를 기반으로 유사한 의료 Concept를 검색하도록 구현하였다. 그림과 같이 "high blood sugar"와 같은 자연어 질의에 대해서도 의미적으로 유사한 의료 Concept가 높은 순위로 검색되는 것을 확인하였다.

```

=====
Query : high blood sugar
=====
[Rank 1]
Score      : 0.9000796
Concept ID  : 444780001
Term       : High glucose level in blood
DescType   : Synonym
Ontology    : SNOMED
-----
[Rank 2]
Score      : 0.85728574
Concept ID  : 68256003
Term       : Increased glucose level
DescType   : Synonym
Ontology    : SNOMED

```

#### 5-2-4. Keyword Search

전처리한 Description 데이터를 이용하여 BM25 기반 Keyword Search를 구현하였다.

BM25 인덱스를 구축한 후 사용자 질의를 입력하면 관련성이 높은 의료 Concept를 순위별로 검색하도록 구현하였으며, 동일한 Concept가 여러 Description으로 검색되는 경우에는 중복을 제거하여 최종 결과를 반환하도록 하였다.

그림과 같이 "HbA1c" 질의에 대해 관련 Concept가 높은 순위로 검색되는 것을 확인하였다.

```
query = "HbA1c"
```

```

=====
[Rank 1]
Score      : 14.794805548754379
Concept ID  : 408591000
Term       : HbA1c target
DescType   : Synonym
Ontology    : SNOMED
-----
[Rank 2]
Score      : 13.261030893604554
Concept ID  : 96595-4
Term       : HbA1c MFr DBS
DescType   : Synonym
Ontology    : LOINC

```

#### 5-2-5. Hybrid Search

Embedding Search와 Keyword Search의 결과를 결합하는 Hybrid Search를 구현하였다.

동일한 질의에 대해 두 검색 결과를 통합한 후 Min-Max 정규화를 수행하고, 가중치를 적용한 Hybrid Score를 계산하여 최종 순위를 결정하도록 구현하였다. 현재는 검색 성능 향상을 위해 가중치 조정 및 성능 평가를 진행하고 있다.

```
query = "type 2 diabetes"
```

|   | conceptId | term                              | ontology | descriptionType | qdrant_score | bm25_score | qdrant_score_norm | bm25_score_norm | hybrid_score |
|---|-----------|-----------------------------------|----------|-----------------|--------------|------------|-------------------|-----------------|--------------|
| 0 | 44054006  | Type 2 diabetes mellitus          | SNOMED   | Synonym         | 0.928106     | 18.183072  | 1.000000          | 1.000000        | 1.000000     |
| 1 | 81531005  | Type 2 diabetes mellitus in obese | SNOMED   | Synonym         | 0.863796     | 0.000000   | 0.930708          | 0.000000        | 0.744567     |

### 5-2-6. Entity Linking (진행 중)

Mention Detection을 통해 추출한 의료 개체에 대해 Hybrid Search를 수행하여 후보 Concept를 검색하는 기능을 구현 중이다.

현재는 Hybrid Search 결과를 기반으로 Top-3 Concept를 선정하여 Concept Set을 생성하는 과정까지 구현하는 중이며, 생성된 Concept Set은 이후 Neo4j Graph Query의 입력으로 활용될 예정이다.

## 5-3. Neo4j 기반 Graph 구축

### 5-3-1. Neo4j Schema 및 Node 구축

의료 질의응답에 활용할 지식 그래프를 구축하기 위해 SNOMED CT, RxNorm, LOINC 데이터를 전처리하여 Neo4j에 적재하였다. 모든 노드는 (:Concept {ontology, concept\_id, name, semantic\_type}) 형태의 공통 스키마를 기반으로 구축하였으며, 각 온톨로지의 특성을 반영한 속성을 함께 저장하였다. 또한 (ontology, concept\_id) 복합키에 유니크 제약 조건을 설정하고, semantic\_type 속성에는 인덱스를 생성하였다.

#### (1) SNOMED CT

전체 529,801개의 개념 중 활성 상태의 Concept을 대상으로 **379,283개의 노드**를 적재하였다. 이름은 FSN(Full Specified Name)을 우선 사용하고, 누락된 경우에는 Synonym으로 보완하였다. 또한 SNOMED 태그를 기준으로 Disorder, Finding, Procedure, BodyStructure, Organism의 5개 semantic\_type으로 분류하였으며, 그 외 개념은 Other로 처리하였다. 활성 관계만을 대상으로 총 **1,331,550개의 Relationship**을 구축하였다.

## (2) RxNorm

원본 244,862행 중 출처(SAB) 우선순위(RXNORM → MTHSPL → MTHCMSFRF)를 기준으로 대표 개념(RXCUI) **82,429개**를 선별하였다. TTY의 동의어 속성(SY, TMSY, PSN)을 제외하여 대표 개념만 남기고, STR 값을 이름으로 매핑하여 노드를 구축하였다. 또한 TTY를 기준으로 Drug, DoseForm, MedicalSupply의 semantic\_type을 부여하였다.

## (3) LOINC

LOINC-SNOMED 확장판 개념 46,057개 중 활성 상태이면서 LOINC 코드와 매핑 가능한 **44,981개의 노드**를 적재하였다. 모든 노드의 semantic\_type은 LabTest로 지정하였으며, 이름은 FSN과 Synonym을 우선 사용하고, 이름이 없는 경우에는 LOINC의 Long Common Name으로 보완하였다.

| Ontology  | 적재 Node 수      |
|-----------|----------------|
| SNOMED CT | 379,283        |
| RxNorm    | 82,429         |
| LOINC     | 44,981         |
| <b>합계</b> | <b>506,693</b> |

최종적으로 Neo4j에 총 506,693개의 Concept Node를 구축하였으며, 이를 Graph Query의 기반 데이터로 활용한다.

### 5-3-2. HAS\_DRUG 관계 구축

UMLS의 개념 간 관계(MRREL) 중 MED-RT의 may\_treat 및 may\_be\_treated\_by 관계를 활용하여 SNOMED CT와 RxNorm 간 HAS\_DRUG 관계를 구축하였다.

대표 질환 5종과 하위 Concept을 대상으로 UMLS CUI를 매칭하고, 관계가 존재하는 CUI를 중심으로 대표 RxNorm 개념을 연결하였다. 직접 연결된 세부 질환에서 관계를 확인할 수 없는 경우에는 상위 Concept까지 탐색 범위를 확장하여 관계를 구축하였다. 또한 관계가 어떤 질환 그룹에서 생성되었는지 확인할 수 있도록 disease\_group 속성을 함께 저장하였다. 최종적으로 **304개의 HAS\_DRUG 관계**를 구축하였다.



Figure 1. HAS\_DRUG 관계 연결 예시

### 5-3-3. HAS\_TEST 관계 구축

HAS\_DRUG로 연결된 약물을 대상으로 LOINC Part-RxNorm 매핑 데이터를 활용하여 HAS\_TEST 관계를 구축하였다. 활성 상태의 검사만을 대상으로 관계를 생성하였으며, 최종적으로 **201개의 HAS\_TEST 관계**를 구축하였다.

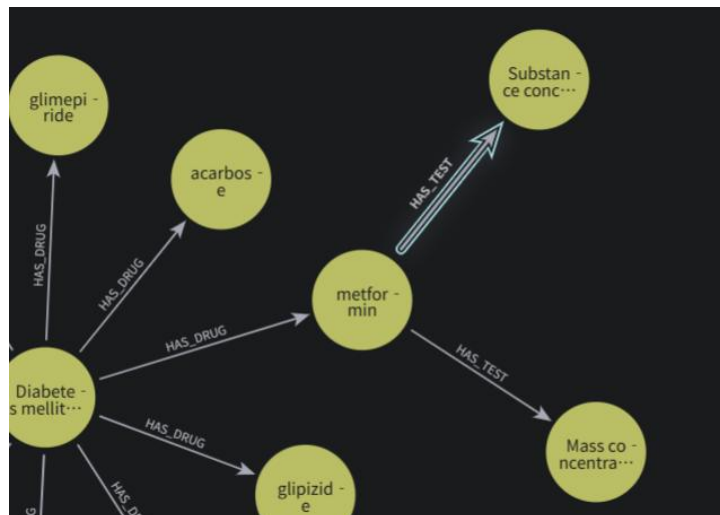


Figure 2. HAS\_TEST 관계 연결 예시

### 5-4. Graph Service 구현

Entity Linking을 통해 생성된 Concept Set을 입력으로 받아 Neo4j에서 관련 의료 지식을 탐색하고, 탐색 결과를 LLM이 활용할 수 있는 형태로 변환하는 Graph Service를 구현하였다. 향후 실험을 통해 점차 구체화할 예정이다.

#### **5-4-1. Graph Query**

Neo4j에서 효율적인 그래프 탐색을 위해 의료적으로 의미 있는 Relationship를 선별하여 Graph Query를 구현하였다. 대표 질환과 관련성이 높은 9개의 Relationship(IS\_A, FINDING\_SITE, DUE\_TO, CAUSATIVE\_AGENT, ASSOCIATED\_MORPHOLOGY, INTERPRETS, OCCURRENCE, HAS\_DRUG, HAS\_TEST)을 대상으로 최대 3-Hop까지 탐색하도록 구성하였다.

#### **5-4-2. Graph Context 생성**

Neo4j에서 조회한 Subgraph를 LLM이 활용할 수 있도록 자연어 형태의 Graph Context 생성 기능을 구현하였다. 각 Relationship에 대해 문장 템플릿을 정의하고, 조회된 노드와 관계 정보를 해당 템플릿에 적용하여 Graph Context를 자동 생성하도록 구성하였다.